

COS 214 Practical 4

Manqoba Mashego – u24589137

Lethukuthula Ngwenya – u24986535

Giselle Mpye - u25622103

GitHub Repository: <https://github.com/u25622103-droid/COS214-PRAC4>

Task 1

Domain: Software Project Management

PNG Tasks is a software project management system for organizing and monitoring software development projects. A project consists of work items arranged in a hierarchy, where task groups can contain individual tasks as well as other task groups. This allows large complex projects to be broken down into smaller manageable units.

Project managers can navigate the hierarchy in different ways to inspect and manage work. Individual tasks follow a lifecycle consisting of states such as Not Started, In Progress, Blocked, and Completed, with their behavior changing depending on their current state. Tasks can also acquire additional responsibilities at such times, such as priority handling, deadline monitoring, and notifications.

GoF Participants:

Iterator:

- ❖ **Iterator:** WorkItemIterator
- ❖ **ConcreteIterator:** DepthFirstIterator, TaskIterator
- ❖ **Aggregate:** WorkItem
- ❖ **ConcreteAggregate:** TaskGroup

Composite:

- ❖ **Component:** WorkItem
- ❖ **Leaf:** Task
- ❖ **Composite:** TaskGroup

State:

- ❖ **State:** TaskState
- ❖ **ConcreteState:** NotStartedState, InProgressState, BlockedState, CompletedState
- ❖ **Context:** Task

Decorator:

- ❖ **Component:** WorkItem
- ❖ **ConcreteComponent:** Task
- ❖ **Decorator:** WorkItemDecorator
- ❖ **ConcreteDecorator:** PriorityDecorator, DeadlineDecorator, NotificationDecorator

Ownership policy:

TaskGroup owns the WorkItems that it contains and is responsible for deleting them. Since TaskGroup may contain other TaskGroup objects, destruction is recursive and destroys the whole hierarchy.

Task owns its current state. A Task is responsible for creating and deleting its current state. When a task transitions to another state, ownership of the previous state is released, and the task now owns the new state.

Decorators own the WorkItems they wrap. A decorator is responsible for deleting its WorkItem. This allows the decorators to be stacked, and each decorator owning the decorator or task it is wrapping.

Iterators do not own the WorkItems they traverse, they only maintain pointers to objects in the hierarchy.

Reflection:

Giselle worked on implementing the code for Decorator and the Iterator, and created the README.md, the git history, the Dockerfile and a bit of the main

Manqoba worked on this PDF, which includes the description of the domain of our project, and also made most of the UML, the makefile, and one of the activity diagrams

Lethukuthula worked on most of the main, the rest of the code besides Iterator and Decorator, two of the activity diagrams, and the state and object diagrams

Valgrind evidence:

```
10 #include "WorkItemDecorator.h"
11
12 void printSeparator()
13 {
14     std::string separator;
15
16     for (int i = 0; i < 80; i++)
17     {
18         separator += "~";
19     }
20 }
```

Problems 14 Output Debug Console Spell Checker 14 Terminal Ports

```
master_p@GoingMerry:~/cos214/COS214-PRAC4$ make valgrind
====>Actual: 0h

Project memory released successfully.
~~~~~
TaskForge terminated successfully.
~~~~~
==764==
==764== HEAP SUMMARY:
==764==    in use at exit: 0 bytes in 0 blocks
==764== total heap usage: 407 allocs, 407 frees, 91,368 bytes allocated
==764==
==764== All heap blocks were freed -- no leaks are possible
==764==
==764== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
○ master_p@GoingMerry:~/cos214/COS214-PRAC4$
```